

# UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería  
Escuela de Ciencias y Sistemas



## Proyecto - Fase 3 Inteligencia Artificial

G#7

| NOMBRE                              | CARNET    |
|-------------------------------------|-----------|
| Vania Argueta Rodríguez             | 201213487 |
| Henry Ronely Mendoza Aguilar        | 202004810 |
| David Eduardo López Morales         | 201907483 |
| Vernik Carlos Alexander Yaxon Ortiz | 201712057 |

# Manual Técnico

## Introducción

Este manual describe en detalle la arquitectura, herramientas y metodologías utilizadas en el desarrollo del proyecto para el curso de Inteligencia Artificial 1 en la Universidad de San Carlos de Guatemala. El propósito del proyecto es implementar un modelo de inteligencia artificial capaz de interpretar entradas de texto en español y responder de forma coherente. Además, incluye la documentación necesaria para la instalación, configuración, y funcionamiento del sistema.

## Objetivos

Desarrollar un modelo que interprete entradas de texto en idioma español y genere respuestas coherentes. Este modelo debe ejecutarse completamente en el navegador, sin depender de servicios externos.

## Herramientas utilizadas

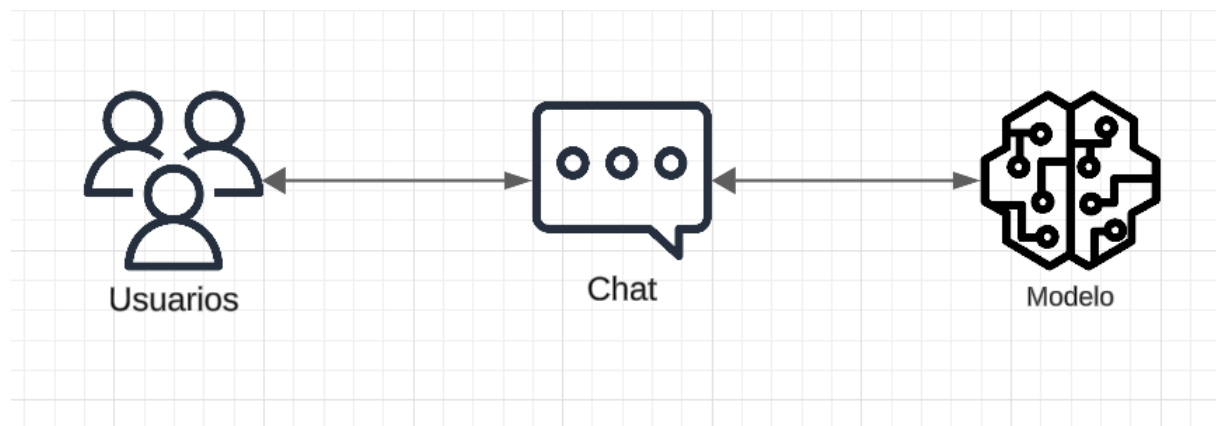
- **Tkinter:** Para esta fase se utilizó la librería tkinter que es una herramienta que nos ayuda para la interfaz gráfica, es una herramienta bastante versátil para esta fase se intentó realizar un diseño sobrio y entendible para todos.
- **Python:** El lenguaje principal del proyecto, Python, permitió la implementación de la lógica de negocio y la interacción entre el modelo de inteligencia artificial y la interfaz gráfica. Su uso garantizó una integración fluida con las bibliotecas Tkinter y TensorFlow.
- **TensorFlow:** TensorFlow es una biblioteca para el desarrollo y entrenamiento de modelos de machine learning directamente en el navegador. En este proyecto, se utilizó para crear un modelo de red neuronal capaz de procesar texto en español y generar respuestas coherentes.

# Arquitectura de la aplicación

El sistema tiene una arquitectura basada en tres capas principales:

1. **Capa de Presentación:** Diseñada con Tkinter y estilizada con Python para proporcionar una experiencia de usuario fluida y atractiva.
2. **Capa de Procesamiento:** Implementada con TensorFlow para procesar y generar respuestas basadas en entradas de texto.
3. **Capa de Interacción:** Gestionada por Python para vincular la lógica del modelo con la interfaz gráfica del chat.

## Flujo de interacción



Este diagrama representa el flujo de interacción en el sistema de chat basado en el modelo previamente entrenado

1. Los Usuarios envían un mensaje al sistema a través del Chat.
2. El Chat pasa la consulta al Modelo para su procesamiento.
3. El Modelo devuelve la respuesta más relevante al Chat.
4. Finalmente, el Chat presenta la respuesta generada a los usuarios.

## Implementación del Modelo

El modelo de inteligencia artificial utilizado en este proyecto se basa en el Sequence To Sequence (Seq2Seq) proporcionado por TensorFlow. Este modelo el cual se debe entrenar con intenciones permite utilizar para realizar tareas de procesamiento de lenguaje natural, como la clasificación o búsqueda de similitudes. Se implementó el modelo en Python y para ser más precisos utilizamos un cuaderno de Colab para poder aportar ideas todos.

## Importación de Librerías Necesarias

```
from ast import literal_eval
import json
import string
import random
import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from keras.layers import Dense, Embedding, LSTM, Input, Flatten
from keras.models import Model, load_model
from sklearn.preprocessing import LabelEncoder
from deep_translator import GoogleTranslator
```

**Definición de una Base de Conocimiento:** Una lista de respuestas predefinidas se almacena en un array, y estas respuestas se comparan con la entrada del usuario:

```

{
  "intents": [
    {
      "tag": "greeting",
      "patterns": [
        "Hi", "Hello", "Good day", "How are you", "Is anyone there?", "Hola", "Buenas", "Que tal"
      ],
      "responses": {
        "en": [
          "Hello, thanks for visiting",
          "Good to see you again",
          "Hi there, how can I help?"
        ],
        "es": [
          "¡Hola!",
          "¡Qué tal!",
          "¿En qué puedo ayudarte?"
        ]
      }
    },
    {
      "tag": "goodbye",
      "patterns": [
        "Bye", "Goodbye", "See you later", "Good day", "Adios", "Hasta luego", "Nos vemos"
      ],
      "responses": {
        "en": [
          "See you later, thanks for visiting",
          "Have a nice day",
          "Bye! Come back again"
        ],
        "es": [
          "¡Adios!",
          "¡Hasta luego!"
        ]
      }
    }
  ]
}

```

**Carga del Modelo y Generación de Respuestas:** El modelo Universal Sentence Encoder se carga de forma asíncrona. Una vez cargado, codifica las oraciones de entrada y las respuestas para comparar similitudes:

```

# Cargar datos
intents_file = "intents3.json" # Cambia esta ruta según corresponda
model_file = "model.h5"        # Cambia esta ruta según corresponda

with open(intents_file, 'r', encoding='utf-8') as content:
    data = json.load(content)

# Preprocesar los datos
print("[INFO] Preprocesando datos...")
tags = []
patterns = []
for intent in data['intents']:
    responses[intent['tag']] = intent['responses']
    for line in intent['patterns']:
        patterns.append(line)
        tags.append(intent['tag'])

# Crear el DataFrame para procesamiento adicional
df = pd.DataFrame({"patterns": patterns, "tags": tags})
df['patterns'] = df['patterns'].apply(
    lambda wrd: [ltrs.lower() for ltrs in wrd if ltrs not in string.punctuation])
df['patterns'] = df['patterns'].apply(lambda wrd: ''.join(wrd))

```

Guardamos el modelo generado y entrenado en python como un archivo tensorflow con estructura json y luego cargarlo en la parte del chat mas adelante.

```

model.compile(loss='sparse_categorical_crossentropy',
              optimizer='adam', metrics=['accuracy'])

# Entrenar y guardar el modelo
model.fit(x_train, y_train, epochs=200)
model.save('model.h5')

```

**Carga del Modelo Generado en Python:** Ahora luego de tener el modelo entrenado y compilado en python procedemos a cargarlo en memoria desde otro archivo Python y poder utilizarlo con nuestra interfaz gráfica.



```

# Cargar datos
intents_file = "intents3.json" # Cambia esta ruta según corresponda
model_file = "model.h5"        # Cambia esta ruta según corresponda

with open(intents_file, 'r', encoding='utf-8') as content:
    data = json.load(content)

```

Generamos un método para obtener las respuestas del modelo ya cargado según los inputs que el usuario genere Y por último generamos una respuesta predictiva según el input del usuario.

```

def responder(mensaje):
    if not mensaje.strip():
        return "No entendí el mensaje, ¿puedes intentarlo de nuevo?"

    # Detectar el idioma del usuario
    idioma_detectado = detectar_idioma(mensaje)
    print(f"[DEBUG] Idioma detectado: {idioma_detectado}")

    # Preprocesar el mensaje
    mensaje_procesado = ''.join([ltrs.lower() for ltrs in mensaje if ltrs not in string.punctuation])
    texts_p = [mensaje_procesado]
    prediction_input = tokenizer.texts_to_sequences(texts_p)
    prediction_input = pad_sequences(prediction_input, maxlen=input_shape)

    # Validar el contenido de la entrada
    if prediction_input.shape[0] == 0:
        return "No entendí el mensaje, intenta escribir algo más claro."

    try:
        # Predecir y obtener el intent
        output = model.predict(prediction_input)
        output = output.argmax()
        response_tag = le.inverse_transform([output])[0]
    except Exception as e:
        print(f"[ERROR] Error al predecir respuesta: {e}")
        return "Lo siento, no puedo procesar tu mensaje en este momento."

    # Seleccionar la respuesta en el idioma correcto
    response_options = responses.get(response_tag, {})
    if idioma_detectado in response_options:
        response = random.choice(response_options[idioma_detectado])

```